

TheKnife

Manuale Tecnico

Sviluppato da: Gianluca Melis,
Alessandro Melnyk,
Davide Redemagni,
Simone Zamberletti

Sommario

Introduzione al Progetto	2
Struttura Tecnica dell'Applicazione	3
Relazioni tra le classi	3
Strutture Dati	5
Scelte di Progettazione Architetture	7
Scelte Algoritmiche.....	9
Formato dei file	10

Introduzione al Progetto

L'applicazione sviluppata ha come obiettivo principale la gestione e la fruizione di informazioni relative ai ristoranti, mettendo in comunicazione diretta i ristoratori con gli utenti finali. Essa permette di visualizzare i ristoranti registrati nel sistema, di lasciare recensioni, di gestire i preferiti e di mantenere un'associazione chiara tra ristoratori e locali posseduti.

Dal punto di vista tecnico, il progetto è stato realizzato in Java, con un'architettura orientata agli oggetti che garantisce modularità, riusabilità e facilità di manutenzione. I dati persistenti vengono gestiti attraverso file JSON, scelti per la loro leggerezza e compatibilità con diverse piattaforme, e manipolati tramite la libreria Gson.

Il sistema è stato concepito per essere facilmente estendibile, sia nell'aggiunta di nuove funzionalità (ad esempio sistemi di ricerca avanzata, filtri o statistiche), sia nell'integrazione con altri servizi. Ogni parte dell'applicazione è stata progettata in modo da separare le responsabilità, semplificando così lo sviluppo collaborativo da parte del team.

Struttura Tecnica dell'Applicazione

L'applicazione è organizzata all'interno di un unico package principale denominato **theknife**, che costituisce il namespace logico del progetto. Questa scelta permette di mantenere tutte le classi coese e facilmente gestibili, garantendo al tempo stesso la possibilità di suddividere in futuro il package in sottopackage specializzati qualora il progetto dovesse evolversi e richiedere ulteriori modifiche

All'interno del package sono presenti diverse tipologie di classi, distinte per responsabilità:

- **Classi di dominio:** rappresentano i concetti fondamentali dell'applicazione.
 - Ristorante: entità principale, descrive le caratteristiche di un ristorante (nome, posizione, tipo di cucina, ecc.).
 - Ristoratore: entità che gestisce uno o più ristoranti.
 - Utente: classe base astratta per rappresentare un generico utente della piattaforma.
 - UtenteRegistrato e UtenteNonRegistrato: sottoclassi che estendono Utente, differenziando i comportamenti e i privilegi.
 - Recensione: collega un utente registrato ad un ristorante, contenendo valutazioni e commenti.
 - TipoCucina: enumerazione che categorizza i ristoranti per tipologia culinaria.
 - Ruolo: enumerazione che distingue i privilegi di accesso (ad esempio amministratore, utente semplice).
 - CriterioRicerca: enumerazione che identifica i possibili filtri che si possono applicare
- **Classi di servizio e gestione:** forniscono metodi per la logica applicativa.
 - Gestione: gestisce le operazioni generali sui dati, come l'inserimento o la ricerca dei ristoranti.
 - OperazioniUtente: definisce le funzionalità legate agli utenti (registrazione, login, interazione con le recensioni).
- **Classi di supporto all'esecuzione:**
 - TheKnife: contiene il metodo main e costituisce il punto di ingresso dell'applicazione. Inoltre, funge da coordinatore tra le varie componenti e integra l'algoritmo di ordinamento mergeSort, utilizzato per la gestione efficiente dei dati.

Relazioni tra le classi

L'architettura logica del sistema è costruita attorno alle seguenti relazioni principali, che descrivono come le entità interagiscono tra loro e come i servizi coordinano le operazioni:

- **Ristorante – Ristoratore**

Ogni oggetto Ristorante è collegato a un Ristoratore, che rappresenta la persona o l'ente responsabile della gestione. La relazione è di tipo uno-a-molti: un singolo ristoratore può infatti possedere e amministrare più ristoranti, mentre ciascun ristorante è riferito a un solo ristoratore. Questo legame consente di implementare funzionalità come la gestione centralizzata dei locali da parte dello stesso proprietario.
- **Ristorante – TipoCucina**

Un ristorante può appartenere a una o più categorie di cucina, modellate tramite l'enumerazione TipoCucina. La relazione è quindi di tipo molti-a-molti, permettendo di rappresentare ristoranti che offrono menù misti o cucine internazionali. Questa

connessione è funzionale ai meccanismi di ricerca e filtraggio, consentendo agli utenti di trovare facilmente i locali in base ai loro interessi gastronomici.

- **Utente – UtenteRegistrato e UtenteNonRegistrato**

Utente costituisce la classe base astratta, da cui derivano UtenteRegistrato e UtenteNonRegistrato. La gerarchia di ereditarietà consente di distinguere i privilegi: un utente registrato dispone di funzionalità avanzate come la scrittura di recensioni o la gestione del proprio profilo, mentre un utente non registrato ha accesso limitato, potendo soltanto consultare i ristoranti e leggere le recensioni esistenti. Questa distinzione garantisce una gestione scalabile dei permessi.

- **UtenteRegistrato – Recensione – Ristorante**

La classe Recensione funge da entità ponte che mette in relazione un UtenteRegistrato con un Ristorante. Ogni recensione contiene un punteggio, un commento e un riferimento all'autore e al locale. La cardinalità è di tipo molti-a-molti mediata da Recensione: un utente registrato può scrivere più recensioni, una per ciascun ristorante visitato, e allo stesso tempo un ristorante può raccogliere recensioni da più utenti.

- **Gestione e OperazioniUtente – Classi di dominio**

Le classi Gestione e OperazioniUtente rappresentano il livello di logica applicativa. Esse non contengono direttamente i dati, ma orchestrano le operazioni sulle entità del dominio. Gestione fornisce metodi per l'inserimento, l'eliminazione e la ricerca dei ristoranti, mentre OperazioniUtente gestisce registrazioni, autenticazioni e interazioni con le recensioni. Entrambe sono progettate per mantenere una separazione chiara delle responsabilità, evitando dipendenze circolari e favorendo la modularità del sistema.

- **TheKnife – Coordinamento generale**

La classe TheKnife costituisce il punto di ingresso del programma e funge da coordinatore tra le varie componenti. Tramite il suo metodo main, essa gestisce l'inizializzazione delle entità e richiama i servizi esposti dalle classi di gestione. Inoltre, integra l'algoritmo mergeSort, utilizzato per ordinare collezioni di dati (ad esempio ristoranti o recensioni) in maniera efficiente. In questo modo, la logica di ordinamento è centralizzata e riutilizzabile all'interno di diversi contesti.

Questa struttura consente una connessione tra le entità principali e i servizi, favorendo estendibilità e manutenibilità.

Strutture Dati

Nel design dell'applicazione "TheKnife", la selezione delle strutture dati è stata guidata da criteri di efficienza, flessibilità e adeguatezza riguardante la gestione di ristoranti, utenti e recensioni. La struttura dati predominante è l'**ArrayList**, una scelta motivata dalla sua capacità di offrire un accesso rapido e indicizzato agli elementi, fondamentale per operazioni di lettura e iterazione. Gran parte dei dati dell'applicazione, come le liste di ristoranti, le recensioni e gli utenti caricati dai file JSON, vengono materializzati in memoria come ArrayList. Questa scelta si rivela ottimale quando le operazioni di lettura e scorrimento sono più frequenti delle operazioni di inserimento e cancellazione in posizioni arbitrarie. Ad esempio, quando un utente non registrato visualizza le recensioni, il sistema carica tutte le recensioni in un ArrayList per poi filtrarle in base ai ristoranti di interesse. Sebbene l'inserimento o la rimozione di elementi al centro della lista possano comportare un costo computazionale lineare, nel contesto dell'applicazione tali operazioni sono meno comuni o il loro impatto è mitigato dalla dimensione generalmente contenuta dei dati gestiti. Inoltre, la natura dinamica dell' ArrayList, che si ridimensiona automaticamente, semplifica la gestione della memoria, astruendo la complessità legata all'allocazione manuale e al rischio di overflow, un vantaggio non trascurabile in un linguaggio gestito come Java.

Un'altra struttura dati cruciale, sebbene utilizzata in un contesto più specifico, è l'**HashMap**, impiegata per la gestione dei ristoranti preferiti degli utenti. Questa scelta è particolarmente strategica: la HashMap permette di associare a ogni utente (identificato da un Integer che rappresenta il suo ID) una lista di ristoranti preferiti (un ArrayList<Integer> contenente gli ID dei ristoranti). L'efficienza di questa struttura risiede nella sua capacità di eseguire operazioni di inserimento, ricerca e rimozione in tempo mediamente costante, $O(1)$. Questa caratteristica è fondamentale per garantire una risposta rapida del sistema quando un utente aggiunge, rimuove o visualizza i propri preferiti, operazioni che devono essere percepite come istantanee per assicurare una buona esperienza utente. L'alternativa, come una lista di oggetti Preferito (contenente ID utente e ID ristorante), avrebbe richiesto una scansione lineare dell'intera lista per recuperare i preferiti di un singolo utente, con un conseguente degrado delle prestazioni all'aumentare del numero di utenti e di preferiti salvati. La HashMap, quindi, rappresenta il compromesso ideale tra utilizzo della memoria e velocità di accesso, ottimizzando una delle funzionalità interattive chiave per l'utente registrato. La combinazione di HashMap e ArrayList per questa funzionalità sfrutta i punti di forza di entrambe le strutture: la prima per l'accesso rapido ai dati di un utente specifico, la seconda per una gestione flessibile e ordinata della lista dei suoi preferiti.

Infine, l'utilizzo delle **enum** (enumerazioni) come Ruolo e CriterioRicerca è una scelta progettuale che migliora significativamente la robustezza e la leggibilità del codice. Le enum forniscono un meccanismo di type-safety, garantendo che variabili come il ruolo di un utente o il criterio di ricerca di un ristorante possano assumere solo un insieme predefinito e limitato di valori. Questo previene errori comuni derivanti dall'uso di costanti stringa o numeriche, come errori di battitura o l'assegnazione di valori non validi, che potrebbero non essere rilevati in fase di compilazione. Nel metodo Registrazione, ad esempio, l'uso di Ruolo.valueOf() permette di convertire in modo sicuro una stringa di input nel tipo Ruolo corrispondente, lanciando un'eccezione IllegalArgumentException se il valore non è valido. Analogamente, nell'implementazione della ricerca, CriterioRicerca guida la logica dello switch, rendendo il codice più pulito, auto-documentante e facile da estendere.

Se in futuro si volesse aggiungere un nuovo criterio di ricerca o un nuovo ruolo utente, sarebbe sufficiente modificare l'enumerazione, e il compilatore Java segnalerebbe tutti i punti del codice (come gli switch) che necessitano di essere aggiornati, riducendo il rischio di introdurre bug. Questa scelta, quindi, non solo previene errori, ma promuove anche una progettazione software più manutenibile e scalabile.

Scelte di Progettazione Architetture

L'architettura del software si fonda su principi di **incapsulamento**, **ereditarietà** e **polimorfismo**, con l'obiettivo di creare un sistema modulare, riutilizzabile e facilmente estendibile. Una delle decisioni architetturali più significative è l'introduzione della classe astratta `Utente` e della sua controparte `OperazioniUtente`. Questa scelta definisce un contratto comune per tutte le entità che rappresentano utenti nel sistema, siano essi registrati (`UtenteRegistrato`, `Ristoratore`) o non registrati (`UtenteNonRegistrato`). La classe `Utente` astrae gli attributi comuni come nome, cognome e username, e implementa logiche di validazione di base (es. `isNomeValido`, `isPasswordValida`), promuovendo il riutilizzo del codice e garantendo coerenza nel trattamento dei dati anagrafici. L'ereditarietà permette alle sottoclassi di specializzare il comportamento: `UtenteRegistrato`, ad esempio, estende `Utente` aggiungendo funzionalità specifiche come la gestione delle recensioni e dei preferiti. La classe astratta `OperazioniUtente`, invece, definisce un'interfaccia di operazioni comuni, come `cercaRistorante` e `visualizzaRecensioni`. L'implementazione di `cercaRistorante` come metodo `final` in questa classe assicura che la logica di ricerca sia standardizzata e non possa essere sovrascritta, garantendo un comportamento uniforme in tutta l'applicazione. Al contrario, `visualizzaRecensioni` è un metodo astratto, costringendo ogni sottoclasse a fornire una propria implementazione, riconoscendo che il modo in cui un utente registrato e uno non registrato accedono alle recensioni può differire. Questa struttura gerarchica non solo organizza il codice in modo logico, ma facilita anche l'introduzione di nuovi tipi di utente in futuro con un impatto minimo sul sistema esistente.

Un'altra scelta architetturale chiave è l'utilizzo di classi di utilità (Utility Classes) annidate e statiche all'interno della classe contenitore `Gestione`. Questo pattern di progettazione, noto come "helper class", raggruppa funzionalità correlate e trasversali in un unico punto, migliorando l'organizzazione e la coesione del codice. La classe `Gestione` funge da namespace per diverse responsabilità: `Gestione.Sort` incapsula l'algoritmo di ordinamento, `Gestione.CifraturaUtils` gestisce la crittografia e la decrittografia, mentre `Gestione.Serializer` e `Gestione.Deserializer` si occupano della persistenza dei dati su file JSON. Dichiarare queste classi come static e annidate all'interno di `Gestione` previene l'inquinamento dello spazio dei nomi globale e chiarisce che queste classi non mantengono uno stato proprio, ma offrono solo metodi di servizio. Ad esempio, `CifraturaUtils` fornisce metodi statici `cripta` e `decripta` che operano sui dati passati come parametri, senza necessità di istanziare un oggetto. Questa centralizzazione delle utility semplifica la manutenzione: se si dovesse cambiare l'algoritmo di crittografia o la libreria di serializzazione, le modifiche sarebbero confinate a una singola classe. Sebbene l'implementazione di `Sort` sia attualmente specializzata per gli oggetti `Ristorante`, la sua struttura generica (`<T>`) è un esempio di lungimiranza progettuale, poiché pone le basi per un suo futuro riutilizzo con altri tipi di dati senza richiedere modifiche sostanziali all'architettura.

La gestione della persistenza dei dati attraverso la **serializzazione e deserializzazione JSON** con la libreria `Gson` di Google rappresenta una scelta fondamentale per l'interoperabilità e la manutenibilità del sistema. JSON è un formato testuale, leggibile dall'uomo e indipendente dal linguaggio, il che facilita il debug e l'eventuale interazione con altri sistemi (es. un'applicazione web o mobile). L'architettura prevede classi dedicate, `Serializer` e `Deserializer`, che astraggono le complessità della conversione tra oggetti Java e la loro rappresentazione JSON. Un aspetto notevole di questa implementazione è l'uso di deserializzatori customizzati, come `Utente.UtenteDeserializer` e `Recensione.RecensioneDeserializer`. Questa tecnica si rivela

indispensabile quando la struttura del JSON non mappa direttamente a un singolo costruttore di classe. Nel caso di `Utente`, che è una classe astratta, non può essere istanziata direttamente. Il deserializzatore custom ispeziona il campo "ruolo" all'interno dell'oggetto JSON e, in base al suo valore (`CLIENTE` o `RISTORATORE`), decide quale sottoclasse concreta (`UtenteRegistrato` o `Ristoratore`) istanziare. Questo approccio permette di mantenere una singola lista di utenti nel file `utenti.json` pur rappresentando tipi di utenti diversi, un esempio elegante di polimorfismo applicato alla persistenza dei dati. Questa scelta architetturale garantisce un disaccoppiamento tra la logica di business e il formato di persistenza, rendendo il sistema più robusto e flessibile ai cambiamenti.

Scelte Algoritmiche

Dal punto di vista algoritmico, il sistema implementa due logiche principali che meritano un'analisi approfondita: l'algoritmo di ordinamento e quello di ricerca. Per l'ordinamento, è stato scelto l'algoritmo Merge Sort, implementato nella classe Gestione.Sort. Questa è una decisione tecnica ponderata, motivata dalle caratteristiche intrinseche del Merge Sort. A differenza di altri algoritmi di ordinamento semplici come il Bubble Sort o l'Insertion Sort, il Merge Sort garantisce una complessità temporale nel caso peggiore di $O(n \log n)$, dove n è il numero di elementi da ordinare. Questa performance stabile e prevedibile è cruciale in un'applicazione interattiva, dove tempi di risposta lenti e imprevedibili possono compromettere l'esperienza utente. Anche se algoritmi come il Quick Sort hanno una complessità media di $O(n \log n)$, il loro caso peggiore degrada a $O(n^2)$, un rischio che il Merge Sort evita. L'implementazione fornita è di tipo ricorsivo e non "in-place", in quanto crea delle sotto-liste (left e right) ad ogni chiamata ricorsiva. Questo approccio, sebbene possa comportare un maggiore utilizzo di memoria rispetto a una versione "in-place" (che opera direttamente sull'array originale), semplifica notevolmente la logica di implementazione e riduce il rischio di errori. La stabilità del Merge Sort (mantiene l'ordine relativo degli elementi uguali) potrebbe essere un ulteriore vantaggio, sebbene non esplicitamente sfruttato nell'uso attuale con un Comparator generico. La scelta del Merge Sort, quindi, privilegia la stabilità delle prestazioni e la robustezza rispetto a un'ottimizzazione estrema della memoria, una scelta sensata per la maggior parte delle applicazioni che non gestiscono dataset di dimensioni astronomiche.

Per quanto riguarda la ricerca dei ristoranti, il metodo cercaRistorante nella classe OperazioniUtente implementa un algoritmo di **ricerca lineare con filtraggio multi-criterio**. Questo approccio, sebbene semplice, è efficace per il contesto applicativo. L'algoritmo itera attraverso l'intera lista di ristoranti fornita come input e, per ciascun ristorante, verifica se soddisfa tutti i criteri specificati nella mappa criteri. La logica è implementata in modo tale che un ristorante viene escluso non appena uno dei criteri non è soddisfatto (grazie alla variabile booleana corrisponde e all'istruzione break). Questa tecnica, nota come "short-circuiting", ottimizza leggermente il processo, evitando controlli superflui su un ristorante che è già stato scartato. La complessità di questo algoritmo è lineare, $O(n \cdot m)$, dove n è il numero di ristoranti nella lista e m è il numero di criteri di ricerca. Data la probabile dimensione contenuta della lista di ristoranti che un'applicazione di questo tipo gestisce in memoria in un dato momento, questa performance è del tutto accettabile. Alternative più complesse, come la creazione di indici su determinati campi (es. un HashMap per la ricerca per città), potrebbero velocizzare le ricerche su singoli criteri, ma complicherebbero la gestione delle ricerche multi-criterio e aumenterebbero il consumo di memoria. L'approccio scelto rappresenta quindi un ottimo equilibrio tra semplicità implementativa, flessibilità (è facile aggiungere nuovi CriterioRicerca) e prestazioni adeguate al dominio del problema. La gestione non case-sensitive per i campi testuali come città, nazione e nome, ottenuta tramite equalsIgnoreCase, migliora ulteriormente l'usabilità della funzione di ricerca dal punto di vista dell'utente finale.

Formato dei file

La gestione dei dati dell'applicazione è stata progettata con l'obiettivo di garantire **efficienza, compatibilità e semplicità di integrazione**. Per questo motivo si è scelto di utilizzare file in formato **JSON**, che rappresentano uno standard ampiamente diffuso e facilmente leggibile sia dalle macchine che dagli sviluppatori.

Come anticipato nelle scelte architetturali, la persistenza delle informazioni avviene tramite **serializzazione e deserializzazione JSON**, implementata con la libreria **Gson** di Google. Questo approccio permette di trasformare rapidamente gli oggetti Java in rappresentazioni testuali e viceversa, riducendo la complessità della gestione manuale dei file e favorendo l'estendibilità futura.

I dati dell'applicazione sono suddivisi in **cinque file principali**, ognuno dei quali è dedicato ad un ambito specifico:

- **gestioneRistoranti.json**

Contiene l'associazione tra utenti e ristoranti. È strutturato con due campi principali:

- **fkIdUtente**: rappresenta l'ID del ristorante.
- **fkIdRistorante**: rappresenta l'ID del ristorante.

Grazie a questa struttura è possibile risalire in maniera rapida al proprietario di un ristorante o, viceversa, ottenere la lista completa dei ristoranti posseduti da un determinato ristorante.

- **preferiti.json**

Gestisce la funzionalità dei preferiti per ciascun utente. Per motivi di semplicità ed efficienza, l'accesso e la modifica a questo file avvengono tramite due metodi specializzati, che incapsulano completamente la logica di lettura e scrittura, garantendo coerenza interna.

- **recensioni.json**

Contiene tutte le recensioni effettuate dagli utenti. Ogni recensione è rappresentata da un oggetto JSON con i seguenti campi:

- **idRecensione**: identificatore univoco della recensione.
- **fkIdRistorante**: ID del ristorante a cui si riferisce la recensione.
- **fkIdUtente**: ID dell'utente che ha inserito la recensione.
- **voto**: punteggio assegnato dall'utente.
- **commento**: testo del commento.
- **data**: data in cui è stato effettuato il commento.
- **idRecensionePadre**: campo che permette la gestione di risposte. È impostato a -1 di default, ma nel caso in cui il ristorante decida di rispondere a una recensione, assume il valore dell'ID della recensione di riferimento.

Questa struttura supporta quindi sia recensioni dirette che thread di risposte, rendendo più flessibile la gestione del feedback.

- **ristoranti.json**

Memorizza la lista di tutti i ristoranti presenti nel sistema, comprensiva delle informazioni principali necessarie per la loro identificazione e visualizzazione.

- **utenti.json**

Contiene l'elenco degli utenti registrati, con i dati necessari per l'autenticazione e la gestione delle funzionalità personalizzate.

Grazie a questa suddivisione modulare è possibile garantire una **chiara separazione delle responsabilità** tra i vari ambiti applicativi, semplificando la manutenzione del codice e rendendo più agevole l'eventuale estensione futura con nuovi file o campi.